Universidad Autónoma de TAMAULIPAS

Unidad Académica Multidisciplinaria Mante

MANUAL DE LABORATORIOS

Arquitectura de Equipos Digitales

Unidades 1, 2 y 3

Laboratorios 1 al 12 (Nivel Básico a Intermedio) + Laboratorio Integrador

Ingeniería en Sistemas Computacionales | 2026-1

DR. DANIEL LÓPEZ PIÑA
DRA. AZUCENA CONTRERAS VILLANUEVA
MD. FELIPE SILVA HERNÁNDEZ

Contenido

Exploración de la Arquitectura del Procesador con CPU-Z y lscpu 3
Análisis de la Jerarquía de Memoria y Detección de Cuellos de Botella 5
Pipeline y Rendimiento: Efecto del Paralelismo en la CPU 8
Exploración de la Arquitectura Interna del ATmega328P 11
Comunicación Serial UART: Interfaz entre MCU y PC 14
Sensor I2C: Lectura de Temperatura y Humedad con BMP280 / DHT 17
PWM y Control Analógico: Dimmer de LED y Control de Servo 20
SPI en Acción: Comunicación con Pantalla TFT ILI9341 24
ESP32 como Servidor Web: IoT con WiFi Integrado 28
Benchmark y Comparativa de Arquitecturas x86 vs ARM 32
Diagnóstico de Hardware con SMART y Herramientas del Sistema 35
Virtualización y Análisis de Overhead: Nativo vs VM vs Contenedor 38
Sistema de Monitoreo Inteligente de Infraestructura TI 41

LABORATORIO 01 | NIVEL: BASICO

Exploración de la Arquitectura del Procesador con CPU-Z y lscpu

Unidad: Unidad 1 - Fundamentos | Duración estimada: 90 minutos | Temas: 1.1, 1.2

OBJETIVO

Identificar y documentar los componentes internos de la CPU del equipo disponible, interpretando el significado técnico de cada parámetro mediante herramientas de diagnóstico.

FUNDAMENTO TEORICO

El procesador (CPU) es el componente central de cualquier sistema computacional. Su arquitectura define la forma en que el hardware ejecuta instrucciones. Los parámetros que caracterizan a una CPU son:

- Núcleo (core): unidad de procesamiento independiente dentro del chip. Cada core puede ejecutar instrucciones de forma independiente.
- Thread / Hilo lógico: con HyperThreading (Intel) o SMT (AMD), cada núcleo físico puede ejecutar 2 hilos en paralelo, apareciendo como 2 CPUs lógicas para el sistema operativo.
- Frecuencia base y turbo: la frecuencia base es el ritmo de operación garantizado bajo cualquier condición. La frecuencia turbo es el máximo alcanzable temporalmente cuando la temperatura y el consumo lo permiten.
- Arquitectura del conjunto de instrucciones (ISA): define el lenguaje que el procesador entiende. x86-64 es el ISA dominante en PCs y servidores; ARM64 en móviles y chips Apple Silicon.
- Cache: memoria ultrarrápida integrada en el procesador organizada en niveles (L1 por core, L2 por core o compartida, L3 compartida por todos los núcleos). Reduce la latencia de acceso a datos frecuentes.
- TDP (Thermal Design Power): potencia máxima que el sistema de refrigeración debe disipar. Indicador del consumo energético esperado.

MATERIALES Y HERRAMIENTAS

- PC o laptop con Windows o Linux
- CPU-Z (cpuid.com) para Windows - gratuito
- HWiNFO64 (hwinfo.com) para Windows - gratuito
- En Linux: comandos lscpu, cat /proc/cpuinfo, dmidecode (requiere sudo)
- Hoja de registro para anotar resultados

PROCEDIMIENTO PASO A PASO

1. Instalar CPU-Z en Windows o verificar que lscpu está disponible en Linux (viene preinstalado en la mayoría de distribuciones).
2. Abrir CPU-Z. En la pestaña 'CPU': anotar Name (modelo completo), Code Name (microarquitectura), Max TDP, Core Speed (frecuencia actual), Multiplier, Bus Speed, y el nivel de instrucciones (MMX, SSE, AVX).
3. En la pestaña 'Cache': registrar el tamaño y tipo de L1 Data Cache, L1 Instruction Cache, L2 Cache y L3 Cache.
4. En la pestaña 'Mainboard': anotar el fabricante y modelo de la tarjeta madre, el chipset y la versión del BIOS/UEFI.
5. En la pestaña 'Memory': registrar el tipo de RAM (DDR4/DDR5), frecuencia, tamaño total y numero de slots usados.
6. En Linux ejecutar: 'lscpu' para arquitectura general, 'cat /proc/cpuinfo | grep -E "model name|cpu MHz|cache size|siblings|cpu cores"' para detalles por core.
7. Completar la tabla de resultados con todos los datos obtenidos de ambas herramientas.
8. Buscar el modelo de CPU en ark.intel.com o amd.com/processors para comparar los valores medidos con las especificaciones oficiales.

Código de ejemplo:

```
// Linux: comandos utiles para este laboratorio
lscpu                                       # resumen de arquitectura
cat /proc/cpuinfo | grep 'model name' | uniq
cat /proc/cpuinfo | grep 'cpu cores' | uniq
cat /proc/cpuinfo | grep 'siblings' | uniq  # nucleos logicos
cat /proc/cpuinfo | grep 'cache size' | uniq
nproc                                        # total de CPUs logicas
dmidecode -t processor | grep -E 'Version|Max Speed|Core Count|Thread Count'

// Monitoreo en tiempo real de frecuencia (Linux)
watch -n 1 'cat /proc/cpuinfo | grep MHz'

// Ver flags de extensiones de instrucciones
cat /proc/cpuinfo | grep flags | head -1
```

PREGUNTAS DE REFLEXION Y REPORTE

- ¿Cuántos núcleos físicos y lógicos tiene la CPU analizada? ¿Qué tecnología usa para duplicar los hilos (HT/SMT)?
- ¿Cuál es la diferencia entre la frecuencia base y la frecuencia turbo medida? ¿En qué escenario se alcanza la turbo?
- ¿Cuánto tamaño total de cache L3 tiene el procesador? ¿Como beneficia esto al rendimiento de aplicaciones que usan estructuras de datos grandes?
- Compara los valores obtenidos con las especificaciones oficiales del fabricante. ¿Hay diferencias? ¿A que podrían deberse?
- Si este procesador fuera reemplazado por uno con el doble de núcleos, pero la misma frecuencia, ¿en qué aplicaciones se notaria más la mejora y en cuales no?

LABORATORIO 02 | NIVEL: BASICO

Análisis de la Jerarquía de Memoria y Detección de Cuellos de Botella

Unidad: Unidad 1 - Fundamentos | Duración estimada: 90 minutos | Temas: 1.4

OBJETIVO

Medir empíricamente la diferencia de velocidad entre los niveles de la jerarquía de memoria (cache, RAM, SSD, HDD) y correlacionar los resultados con los conceptos teóricos de latencia y ancho de banda.

FUNDAMENTO TEORICO

La jerarquía de memoria es uno de los conceptos más fundamentales de la arquitectura de computadoras. Cada nivel ofrece un compromiso diferente entre velocidad, capacidad y costo:

- Cache L1 (~1-4 ns): la más rápida, integrada directamente en el core del procesador. Almacena el código y datos más recientemente usados.
- Cache L2 (~5-15 ns): más grande que L1, un poco más lenta. Puede ser exclusiva por core o compartida según el diseño.
- Cache L3 (~30-50 ns): compartida entre todos los cores. Actúa como buffer antes de recurrir a la RAM principal.
- RAM DDR (60-100 ns): la memoria principal del sistema. Mucho más lenta que cualquier nivel de cache, pero de mucha mayor capacidad (GB vs MB).
- SSD NVMe (~100 us): almacenamiento persistente de alta velocidad. 1000x más lento que la RAM, pero no volátil.
- HDD (~5-10 ms): almacenamiento magnético. 100,000x más lento que la RAM. Persistente.

El principio de localidad es la razón por la que el cache funciona: localidad temporal (datos usados recientemente se usaran de nuevo) y localidad espacial (datos cercanos en memoria a los usados también se usaran).

MATERIALES Y HERRAMIENTAS

- PC con Windows o Linux
- AIDA64 Trial (64 bits) o CacheBench (Linux) para benchmark de cache/RAM
- CrystalDiskMark 8 (Windows) o fio (Linux) para benchmark de almacenamiento
- HWiNFO64 para monitoreo
- Calculadora para análisis de resultados

PROCEDIMIENTO PASO A PASO

1. Instalar CrystalDiskMark 8 en Windows. En Linux, instalar fio: 'sudo apt install fio'.
2. En Windows con AIDA64 (Trial): ir a Tools > System Stability Test; luego abrir el benchmark de memoria en Tools > Memory Benchmark. Ejecutar Read, Write y Copy. Anotar los MB/s para cada operación.
3. Ejecutar CrystalDiskMark: perfil 'Default', tamaño de prueba 1 GB, en el disco principal. Anotar: SEQ1M lectura, SEQ1M escritura, RND4K lectura, RND4K escritura.
4. Si hay un HDD disponible (o una USB 2.0), repetir CrystalDiskMark en ese dispositivo y anotar los mismos valores.
5. En Linux: ejecutar 'sudo hdparm -Tt /dev/sda' (o nvme0n1 para NVMe) para medir cache de disco y velocidad de lectura del dispositivo.
6. Construir una tabla con los resultados: nivel de memoria, velocidad de lectura medida, velocidad teórica esperada, y la diferencia.
7. Calcular el ratio de velocidad entre cada nivel consecutivo (cuantas veces más rápido es el nivel superior).

Código de ejemplo:

```
// Linux: benchmark de memoria con dd (simple pero util)
# Escribir 1 GB a RAM (tmpfs) y medir velocidad
dd if=/dev/zero of=/tmp/testfile bs=1G count=1 oflag=dsync

# Leer desde cache del disco (probable L3/RAM)
echo 3 > /proc/sys/vm/drop_caches  # limpiar cache primero (sudo)
dd if=/tmp/testfile of=/dev/null bs=1G count=1

// Linux: benchmark de disco con fio
# Lectura secuencial
fio --name=seq-read --rw=read --bs=1M --size=1G --runtime=30 --output-
format=normal

# Escritura aleatoria 4K (simula IOPS de base de datos)
fio --name=rand-write --rw=randwrite --bs=4k --size=1G --runtime=30 --
iodepth=32

// Windows: benchmark de memoria con PowerShell (dato rapido)
Measure-Command { $arr = New-Object byte[] 500MB;
[System.IO.File]::ReadAllBytes('C:\testfile') }
```

PREGUNTAS DE REFLEXION Y REPORTE

- ¿Cuántas veces más rápida es la RAM respecto al SSD en lectura secuencial? ¿Y respecto al HDD?
- ¿Por qué la velocidad de lectura aleatoria en 4K es tan inferior a la secuencial, especialmente en HDD?

- Si una aplicación de base de datos accede frecuentemente a un índice de 50 MB, ¿cabria en la cache L3 del procesador analizado en el Lab 1? ¿Qué implicaciones tiene esto para el rendimiento?
- ¿Qué tipo de aplicación se beneficiaría más de pasar de HDD a SSD NVMe: una que lee archivos de 1 GB secuencialmente o una base de datos con transacciones aleatorias de 4KB? Justifica.
- Dibuja una pirámide de la jerarquía de memoria con los valores reales medidos en este laboratorio.

LABORATORIO 03 | NIVEL: BASICO

Pipeline y Rendimiento: Efecto del Paralelismo en la CPU

Unidad: Unidad 1 - Fundamentos | Duración estimada: 90 minutos | Temas: 1.5

OBJETIVO

Demostrar empíricamente el impacto del procesamiento paralelo y la ejecución multihilo en el rendimiento del sistema, comparando rendimiento single-core vs multi-core y midiendo el efecto de la Ley de Amdahl.

FUNDAMENTO TEORICO

El procesamiento paralelo permite ejecutar múltiples tareas simultáneamente. En hardware moderno, esto se logra mediante:

- Pipeline: divide la ejecución de instrucciones en etapas (fetch, decode, execute, memory, writeback). Mientras una instrucción está en la etapa 'execute', la siguiente ya está en 'decode'. Esto permite CPI ~ 1 en condiciones ideales.
- Procesadores multicore: cada core tiene su propio pipeline completo. Con N cores, el sistema puede ejecutar N instrucciones completamente independientes en paralelo real.
- Hyper-Threading / SMT: un core físico ejecuta 2 hilos de forma pseudo paralela, alternando entre ellos cuando uno espera datos de memoria. Mejora la utilización del pipeline pero no duplica el rendimiento.
- Ley de Amdahl: establece el límite de aceleramiento con paralelismo. Si el 30% del programa es secuencial (no paralelizable), el máximo speedup posible con N cores es 1/0.30 = 3.33x, sin importar cuantos cores se agreguen.

Los benchmarks de CPU se dividen en: Single-Core (mide el rendimiento de un solo hilo, relevante para aplicaciones no paralelizadas) y Multi-Core (mide el rendimiento de todos los hilos en paralelo, relevante para servidores y aplicaciones optimizadas).

MATERIALES Y HERRAMIENTAS

- PC con Windows o Linux con procesador multicore (mínimo 4 cores recomendado)
- Cinebench 2024 (gratuito, maxon.net/cinebench) o Cinebench R23
- Prime95 (Windows/Linux) para prueba de estrés
- HWiNFO64 para monitoreo de temperatura y frecuencia durante las pruebas
- Opcional: Handbrake (encoder de video) para benchmark real

PROCEDIMIENTO PASO A PASO

1. Instalar Cinebench 2024 y HWiNFO64. Antes de cualquier prueba, cerrar todas las aplicaciones abiertas para resultados limpios.

2. En HWiNFO64: iniciar el monitoreo de sensores (botón 'Sensors' en la ventana principal). Dejar visible en segundo plano.
3. Ejecutar Cinebench 2024 - prueba Single-Core (dura ~10 minutos). Anotar: puntaje npts, temperatura máxima del CPU, frecuencia máxima alcanzada durante la prueba.
4. Esperar 5 minutos para que el CPU se enfríe, luego ejecutar Cinebench 2024 - prueba Multi-Core (dura ~10 minutos). Anotar: puntaje npts, temperatura máxima, frecuencia promedio y consumo de paquete (package power en Watts si HWiNFO lo muestra).
5. Calcular el ratio Multi-Core / Single-Core. Comparar con el número de núcleos lógicos del sistema.
6. Calcular la eficiencia de escala: (ratio Multi/Single) / (núcleos lógicos) x 100%. ¿Qué porcentaje del máximo teórico se alcanza?
7. Buscar los resultados del mismo CPU en cinebench.maxon.net para comparar con la media global y con otros procesadores.
8. OPCIONAL - Handbrake: codificar un video de 10 minutos a H.265 con todos los núcleos. Luego en configuración avanzada limitar a 1 thread y repetir. Medir el tiempo de cada prueba.

Código de ejemplo:

```
// Python: demostrar la Ley de Amdahl numericamente
# Instalar: pip install matplotlib (opcional para grafica)
def speedup_amdahl(p_paralelo, n_cores):
    # p_paralelo: fraccion del programa que puede paralelizarse (0.0 a 1.0)
    # n_cores: numero de procesadores/cores
    p_serial = 1 - p_paralelo
    return 1 / (p_serial + p_paralelo / n_cores)

# Ejemplo: un programa 80% paralelizable
fracciones = [0.5, 0.75, 0.90, 0.95]
cores = [1, 2, 4, 8, 16, 32, 64]

for frac in fracciones:
    print(f'\nFraccion paralela: {frac*100:.0f}%')
    for n in cores:
        sp = speedup_amdahl(frac, n)
        print(f'  {n:3d} cores -> speedup: {sp:.2f}x')

# Limite teorico con cores infinitos:
for frac in fracciones:
    limite = 1 / (1 - frac)
    print(f'Limite maximo con {frac*100:.0f}% paralelo: {limite:.2f}x')
```

PREGUNTAS DE REFLEXION Y REPORTE

- ¿Cuánto fue el ratio Multi-Core/Single-Core en Cinebench 2024? ¿Se acerca al número de núcleos lógicos? ¿Por qué hay una diferencia?

- Aplicando la Ley de Amdahl con el ratio obtenido, ¿qué porcentaje aproximado del benchmark de Cinebench es paralelizable?
- ¿Qué efecto tuvo el calor en la frecuencia del CPU durante la prueba Multi-Core? ¿Se observo algun throttling (reducción de frecuencia por temperatura)?
- Según la Ley de Amdahl, si una aplicación es 95% paralelizable, ¿cuál es el máximo speedup posible con 128 cores? ¿Vale la pena invertir en tantos cores para esta aplicación?
- ¿En qué tipo de aplicaciones empresariales o de ingeniería importa más el rendimiento Single-Core? ¿Y el Multi-Core?

LABORATORIO 04 | NIVEL: BASICO

Exploración de la Arquitectura Interna del ATmega328P

Unidad: Unidad 2 - Microcontroladores | Duración estimada: 100 minutos | Temas: 2.1, 2.2

OBJETIVO

Leer e interpretar el datasheet del ATmega328P, identificar su arquitectura Harvard, mapa de memoria y registros, y verificar el funcionamiento básico del MCU mediante el primer programa en Arduino IDE.

FUNDAMENTO TEORICO

El ATmega328P es un microcontrolador RISC de 8 bits con arquitectura Harvard modificada de Atmel/Microchip. Es la base del Arduino UNO y una de las plataformas de aprendizaje más usadas del mundo.

- CPU de 8 bits con 32 registros de propósito general (R0-R31). Los pares R26:R27, R28:R29 y R30:R31 forman los registros de puntero X, Y y Z para direccionamiento indirecto de 16 bits.
- Memoria Flash de 32 KB para almacenar el programa (no volátil). El bootloader de Arduino ocupa los últimos 2 KB, dejando 30 KB disponibles para el usuario.
- SRAM de 2 KB para variables en ejecución, stack y heap. Esta memoria es el recurso más critico en proyectos complejos.
- EEPROM de 1 KB para datos persistentes que sobreviven al apagado (configuraciones, contadores, calibraciones).
- Pipeline de 2 etapas: mientras ejecuta la instrucción actual, ya está haciendo fetch de la siguiente. Resultado: la mayoría de las instrucciones se ejecutan en 1 ciclo de reloj.
- Frecuencia máxima: 20 MHz con voltaje de 5V, 10 MHz con 3.3V. El Arduino UNO opera a 16 MHz con cristal externo de cuarzo.

> **Nota:** La arquitectura Harvard del ATmega328P significa que tiene buses de 16 bits separados para instrucciones (Flash) y datos (SRAM), lo que permite leer la siguiente instrucción al mismo tiempo que lee o escribe datos.

MATERIALES Y HERRAMIENTAS

- Arduino UNO o Nano (ATmega328P)
- Cable USB tipo A-B (para UNO) o A-miniB (para Nano)
- Arduino IDE 2.x instalado (arduino.cc/en/software)
- Datasheet ATmega328P descargado de microchip.com
- LED externo y resistencia 220 ohm (para el paso avanzado)
- Protoboard y jumpers

PROCEDIMIENTO PASO A PASO

1. Descargar y abrir el datasheet del ATmega328P desde microchip.com. Localizar la sección 'Block Diagram' (tipicamente pagina 5-7). Identificar todos los bloques y anotarlos.
2. En el datasheet, ir a la sección 'Memory' o 'Memory Organization'. Localizar el mapa de memoria y anotar los rangos de direcciones para: Flash (0x0000 - 0x7FFF), SRAM (0x0100 - 0x08FF) y Registros I/O (0x0020 - 0x00FF).
3. Instalar Arduino IDE 2.x. Conectar el Arduino UNO al PC. En Herramientas > Placa, seleccionar 'Arduino UNO'. En Puerto, seleccionar el COM disponible.
4. Abrir Archivo > Ejemplos > Basics > Blink. Compilar con el botón verificar (palomita). Observar el tamaño del sketch en la consola (ej. 'Sketch uses 924 bytes (3%) of program storage space').
5. Cargar el sketch. Verificar que el LED integrado (pin 13) parpadea cada 1 segundo.
6. Modificar el sketch: cambiar ambos delay(1000) por delay(250). Cargar de nuevo. ¿Que cambio?
7. Abrir el Monitor Serial (Ctrl+Shift+M). Modificar el sketch para incluir 'Serial.begin(9600)' en setup() y 'Serial.println("Ciclo: " + String(millis()))' en loop(). Observar la salida.
8. AVANZADO: agregar un LED externo en el pin 7 con resistencia 220 ohm. Hacer que parpadee en contrafase con el LED del pin 13 (cuando uno enciende el otro apaga).

Código de ejemplo:

```
// sketch01_blink_avanzado.ino
// Dos LEDs en contrafase con reporte serial

const int LED_BUILTIN_PIN = 13;  // LED integrado
const int LED_EXTERNO_PIN = 7;   // LED externo con resistencia 220 ohm
const int INTERVALO_MS = 500;    // periodo en milisegundos

unsigned long ultimoTiempo = 0;
bool estadoLed = false;
unsigned long contadorCiclos = 0;

void setup() {
    Serial.begin(9600);
    pinMode(LED_BUILTIN_PIN, OUTPUT);
    pinMode(LED_EXTERNO_PIN, OUTPUT);
    Serial.println("Sistema iniciado. ATmega328P @ 16MHz");
    Serial.println("SRAM total: 2048 bytes");
}

void loop() {
    unsigned long ahora = millis();
    if (ahora - ultimoTiempo >= INTERVALO_MS) {
        ultimoTiempo = ahora;
        estadoLed = !estadoLed;
        digitalWrite(LED_BUILTIN_PIN, estadoLed ? HIGH : LOW);
```

```
        digitalWrite(LED_EXTERNO_PIN, estadoLed ? LOW : HIGH);  // contrafase
        contadorCiclos++;
        Serial.print("Ciclo #"); Serial.print(contadorCiclos);
        Serial.print(" | Tiempo: "); Serial.print(ahora);
        Serial.print(" ms | LED: "); Serial.println(estadoLed ? "ON" : "OFF");
    }
    // Nota: NO usar delay() - permite multitarea cooperativa
}
```

PREGUNTAS DE REFLEXION Y REPORTE

- ¿Cuántos bytes ocupa el sketch de Blink básico en Flash? ¿Cuánto porcentaje de los 30 KB disponibles es?
- En el mapa de memoria del ATmega328P, ¿por qué los registros I/O están mapeados en el espacio de datos (SRAM) en lugar de un espacio separado?
- ¿Qué diferencia hay entre usar delay(500) y usar millis() para temporizar el parpadeo? ¿Por qué millis() es mejor en aplicaciones reales?
- Si agregas 10 variables int (2 bytes cada una) al sketch, ¿cuánto SRAM adicional consumen? ¿Cuánto SRAM queda disponible para el stack?
- ¿Podrías implementar un sistema de semáforo de tráfico (rojo-amarillo-verde) con los tiempos correctos usando el patrón millis() visto en el código? Bosqueja la lógica.

LABORATORIO 05 | NIVEL: BASICO

Comunicación Serial UART: Interfaz entre MCU y PC

Unidad: Unidad 2 - Microcontroladores | Duración estimada: 100 minutos | Temas: 2.5

OBJETIVO

Establecer comunicación bidireccional UART entre un microcontrolador y una PC, enviando datos estructurados y procesando comandos recibidos desde el monitor serial.

FUNDAMENTO TEORICO

UART (Universal Asynchronous Receiver/Transmitter) es el protocolo de comunicación serial mas simple y ubicuo en sistemas embebidos. Sus características principales:

- Asincrónico: no hay señal de reloj compartida. Ambos extremos deben configurar el mismo baud rate y formato de trama previamente.
- Trama: 1 start bit (siempre 0) + 8 bits de datos + 1 stop bit (siempre 1) = 10 bits por carácter. Formato 8N1 (8 datos, sin paridad, 1 stop).
- Baud rate: numero de bits por segundo. A 9600 bps, 1 carácter tarda ~1.04 ms. A 115200 bps, tarda ~87 us.
- Niveles TTL: HIGH = Vcc (3.3V o 5V), LOW = GND (0V). Para conectar al puerto serie de PC antiguo se requiere MAX232 para conversión a RS-232 (+/-12V).
- En Arduino, la clase Serial abstrae el hardware UART0: Serial.begin(baud) configura el hardware, Serial.print() envia datos, Serial.available() verifica si hay datos recibidos, Serial.read() lee el siguiente byte.

El chip CH340G o CP2102 en las tarjetas Arduino/ESP32 convierte el UART del MCU a USB, permitiendo comunicación con el PC sin necesidad de un puerto serie físico.

MATERIALES Y HERRAMIENTAS

- Arduino UNO/Nano o ESP32 DevKit
- Cable USB
- Arduino IDE 2.x instalado
- Monitor serial integrado en Arduino IDE (Ctrl+Shift+M)
- Opcional: PuTTY o Termite para comunicación serial avanzada

PROCEDIMIENTO PASO A PASO

1. Conectar el Arduino al PC. Abrir Arduino IDE, seleccionar la placa y puerto correctos.
2. Crear un nuevo sketch. Copiar el código de ejemplo de este laboratorio.
3. Compilar y cargar el sketch al Arduino.

4. Abrir el Monitor Serial (Ctrl+Shift+M o Herramientas > Monitor Serial). Configurar la velocidad a 115200 bps y seleccionar 'Ambos NL & CR'.
5. Observar los mensajes de inicio que el Arduino envía automáticamente al arrancar.
6. En el campo de texto del monitor serial, escribir 'LED_ON' y presionar Enter. Verificar que el LED integrado enciende.
7. Escribir 'LED_OFF' y verificar que el LED se apaga.
8. Escribir 'STATUS' y verificar que el Arduino responde con su estado actual.
9. Escribir 'BLINK:3' y verificar que el LED parpadea 3 veces rápido como confirmación.
10. Intentar enviar un comando invalido (ej. 'HOLA') y verificar el mensaje de error.

Código de ejemplo:

```
// lab05_uart_comandos.ino
// Comunicacion UART bidireccional con parser de comandos

const int LED_PIN = 13;
bool ledEstado = false;
String bufferCmd = "";

void setup() {
    Serial.begin(115200);
    pinMode(LED_PIN, OUTPUT);
    // Mensaje de bienvenida con datos del sistema
    Serial.println("==============================");
    Serial.println("Sistema UART - Arduino/ATmega");
    Serial.println("Baud rate: 115200 | Formato: 8N1");
    Serial.println("Comandos: LED_ON, LED_OFF, STATUS, BLINK:N");
    Serial.println("==============================");
}

void loop() {
    // Leer datos disponibles en el buffer UART
    while (Serial.available() > 0) {
        char c = Serial.read();
        if (c == '\n') {
            procesarComando(bufferCmd);
            bufferCmd = "";  // limpiar buffer
        } else if (c != '\r') {
            bufferCmd += c;  // acumular caracteres
        }
    }
}

void procesarComando(String cmd) {
    cmd.trim();  // eliminar espacios al inicio/fin
    Serial.print("[CMD RECIBIDO]: '"); Serial.print(cmd); Serial.println("'");

    if (cmd == "LED_ON") {
        ledEstado = true;
        digitalWrite(LED_PIN, HIGH);
```

```
        Serial.println("[OK] LED encendido");
    }
    else if (cmd == "LED_OFF") {
        ledEstado = false;
        digitalWrite(LED_PIN, LOW);
        Serial.println("[OK] LED apagado");
    }
    else if (cmd == "STATUS") {
        Serial.println("--- STATUS ---");
        Serial.print("LED: "); Serial.println(ledEstado ? "ON" : "OFF");
        Serial.print("Uptime: "); Serial.print(millis()/1000);
Serial.println(" s");
        Serial.print("Free SRAM aprox: "); Serial.print(freeRam());
Serial.println(" bytes");
    }
    else if (cmd.startsWith("BLINK:")) {
        int veces = cmd.substring(6).toInt();
        for (int i = 0; i < veces; i++) {
            digitalWrite(LED_PIN, HIGH); delay(150);
            digitalWrite(LED_PIN, LOW);  delay(150);
        }
        Serial.print("[OK] Blink x"); Serial.println(veces);
    }
    else {
        Serial.print("[ERROR] Comando desconocido: '"); Serial.print(cmd);
Serial.println("'");
    }
}

int freeRam() {
    extern int __heap_start, *__brkval;
    int v;
    return (int) &v - (__brkval == 0 ? (int) &__heap_start : (int) __brkval);
}
```

PREGUNTAS DE REFLEXION Y REPORTE

- ¿Cuántos bits se transmiten por carácter en formato 8N1 a 115200 bps? ¿Cuánto tiempo tarda en transmitirse la cadena 'LED_ON' completa?
- ¿Qué ocurre si el Arduino está configurado a 115200 bps pero el monitor serial está a 9600 bps? Prueba este escenario y describe el comportamiento.
- ¿Por qué se usa un buffer de String para acumular los caracteres del comando en lugar de procesar cada byte individualmente al recibirlo?
- El comando 'BLINK:N' usa delay() dentro del procesador de comandos. ¿Qué problema tiene esto? ¿Como rediseñarías el sistema para que el Arduino pueda recibir otros comandos mientras parpadea?
- ¿Como extenderías el protocolo para que el Arduino envíe datos de sensor cada 2 segundos Y al mismo tiempo procese comandos entrantes? Describe la arquitectura del firmware.

LABORATORIO 06 | NIVEL: INTERMEDIO

Sensor I2C: Lectura de Temperatura y Humedad con BMP280 / DHT

Unidad: Unidad 2 - Microcontroladores | Duración estimada: 120 minutos | Temas: 2.1, 2.5

OBJETIVO

Establecer comunicación I2C entre un microcontrolador y un sensor ambiental, escanear el bus I2C para detectar dispositivos, leer mediciones periódicas y mostrarlas formateadas por UART.

FUNDAMENTO TEORICO

I2C (Inter-Integrated Circuit) es un protocolo de comunicación serial síncrono de 2 cables que permite conectar múltiples dispositivos en el mismo bus. Sus características clave son:

- 2 líneas: SDA (datos bidireccional) y SCL (reloj, generado por el master). Ambas son open-drain y requieren resistencias pull-up a Vcc (típicamente 4.7 kohm).
- Direccionamiento de 7 bits: cada dispositivo en el bus tiene una dirección única (ej. BMP280 puede ser 0x76 o 0x77 dependiendo del pin SDO).
- Transacción I2C: START + dirección de 7 bits + bit R/W + ACK + datos + STOP. El master siempre inicia y controla el reloj.
- ACK/NACK: después de cada byte, el receptor envía un bit de confirmación. Si el slave no responde (NACK), indica error o dispositivo ausente.
- Velocidades estándar: 100 kHz (Standard Mode), 400 kHz (Fast Mode), 1 MHz (Fast-Plus Mode).

El BMP280 de Bosch Sensortec es un sensor de presión barométrica y temperatura con interfaz I2C y SPI. Provee lecturas de temperatura con precisión de +/-0.5 grados C y presión con resolución de 0.01 hPa. Su dirección I2C es 0x76 (SDO=GND) o 0x77 (SDO=Vcc).

> **Nota:** Antes de cualquier comunicación I2C, ejecutar un escáner de bus para confirmar que el dispositivo esta correctamente conectado y su dirección I2C. Un dispositivo que no responde al scanner indica problema de conexión o dirección incorrecta.

MATERIALES Y HERRAMIENTAS

- Arduino UNO/Nano o ESP32 DevKit
- Sensor BMP280 (o BMP180, BME280) con breakout board que incluye pull-ups
- Alternativamente: sensor DHT22 con resistencia pull-up 10 kohm
- Resistencias pull-up 4.7 kohm x2 (si el modulo del sensor no las incluye)
- Protoboard y jumpers macho-macho y macho-hembra
- Arduino IDE 2.x con biblioteca 'Adafruit BMP280 Library' instalada

PROCEDIMIENTO PASO A PASO

1. Conexión del sensor BMP280: VCC -> 3.3V del Arduino, GND -> GND, SDA -> A4 (Arduino UNO) o pin 21 (ESP32), SCL -> A5 (Arduino UNO) o pin 22 (ESP32).
2. Si el módulo no incluye resistencias pull-up, colocar 4.7 kohm entre SDA y 3.3V, y otro entre SCL y 3.3V.
3. En Arduino IDE: ir a Sketch > Include Library > Manage Libraries. Buscar 'Adafruit BMP280' e instalar (instalar también sus dependencias cuando se solicite).
4. Cargar primero el I2C Scanner (código de ejemplo mas abajo) para verificar que el sensor es detectado. Debería reportar la dirección 0x76 o 0x77.
5. Si el scanner detecta el sensor, cargar el sketch principal del laboratorio.
6. Abrir el Monitor Serial a 115200 bps. Verificar que los valores de temperatura son coherentes (temperatura ambiente, ~18-28 grados C).
7. Cubrir el sensor con la mano durante 30 segundos y observar si la temperatura sube. Documentar el cambio.
8. Calcular la altitud a partir de la presión medida (incluida en la biblioteca). Comparar con la altitud real de tu ubicación (buscar en Google Maps).

Código de ejemplo:

```
// ─── I2C Scanner (cargar primero para verificar conexión) ───
#include <Wire.h>

void setup() {
    Wire.begin();
    Serial.begin(115200);
    Serial.println("Escaneando bus I2C...");
    int deviceCount = 0;
    for (byte addr = 1; addr < 127; addr++) {
        Wire.beginTransmission(addr);
        byte error = Wire.endTransmission();
        if (error == 0) {
            Serial.print("Dispositivo en 0x");
            if (addr < 16) Serial.print("0");
            Serial.println(addr, HEX);
            deviceCount++;
        }
    }
    Serial.print("Total dispositivos encontrados: ");
    Serial.println(deviceCount);
}
void loop() {}

// ─── Lectura BMP280 con reporte estructurado ───
#include <Wire.h>
#include <Adafruit_BMP280.h>

Adafruit_BMP280 bmp;
const float PRESION_MAR = 1013.25;  // hPa al nivel del mar
unsigned long ultimaLectura = 0;
```

```
const long INTERVALO = 3000;  // leer cada 3 segundos
int numerolectura = 0;

void setup() {
    Serial.begin(115200);
    Serial.println("Lab 06 - Sensor BMP280 via I2C");
    if (!bmp.begin(0x76)) {  // probar 0x77 si falla
        Serial.println("ERROR: BMP280 no encontrado. Verificar conexion y
direccion I2C.");
        while (1) delay(100);  // detener ejecucion
    }
    // Configuracion: modo normal, oversampling x2
    bmp.setSampling(Adafruit_BMP280::MODE_NORMAL,
                    Adafruit_BMP280::SAMPLING_X2,
                    Adafruit_BMP280::SAMPLING_X16,
                    Adafruit_BMP280::FILTER_X16,
                    Adafruit_BMP280::STANDBY_MS_500);
    Serial.println("BMP280 inicializado correctamente.");
    Serial.println("N | Temp(C) | Presion(hPa) | Altitud(m)");
    Serial.println("--|---------|-------------|----------");
}

void loop() {
    if (millis() - ultimaLectura >= INTERVALO) {
        ultimaLectura = millis();
        numerolectura++;
        float temp   = bmp.readTemperature();
        float presion = bmp.readPressure() / 100.0F;  // Pa -> hPa
        float altitud = bmp.readAltitude(PRESION_MAR);
        Serial.print(numerolectura); Serial.print(" | ");
        Serial.print(temp, 1); Serial.print(" C   | ");
        Serial.print(presion, 2); Serial.print(" hPa | ");
        Serial.print(altitud, 1); Serial.println(" m");
    }
}
```

PREGUNTAS DE REFLEXION Y REPORTE

- ¿En qué dirección I2C fue detectado el sensor? ¿Que determina si es 0x76 o 0x77?
- ¿Cuántos dispositivos podrían conectarse simultáneamente en el mismo bus I2C del Arduino? ¿Qué limitación existe si dos sensores diferentes tienen la misma dirección fija?
- ¿Por qué se usa 'millis()' en lugar de 'delay(3000)' para temporizar las lecturas? Describe una situación donde delay() causaría un problema real.
- La altitud calculada por el sensor, ¿coincide con la altitud real de tu ubicación? ¿Por qué podría haber diferencia? ¿Qué factores afectan la precisión?
- Si quisieras agregar una pantalla OLED SSD1306 (también I2C, dirección 0x3C) al mismo bus, ¿necesitarías cambiar el cableado de SDA/SCL? Describe como conectarlas juntas.

LABORATORIO 07 | NIVEL: INTERMEDIO

PWM y Control Analógico: Dimmer de LED y Control de Servo

Unidad: Unidad 2 - Microcontroladores | Duración estimada: 120 minutos | Temas: 2.1, 2.5

OBJETIVO

Generar señales PWM con un microcontrolador para controlar el brillo de un LED y la posición de un servomotor, comprendiendo la relación entre el ciclo de trabajo y el efecto físico resultante.

FUNDAMENTO TEORICO

PWM (Pulse Width Modulation) es una técnica para generar señales analógicas aproximadas usando salidas digitales. El concepto central es el ciclo de trabajo (duty cycle):

- Duty cycle: porcentaje del tiempo que la señal está en HIGH en un periodo completo. 0% = siempre LOW (0V), 50% = mitad HIGH, 100% = siempre HIGH (Vcc).
- Frecuencia PWM: cuantos ciclos completos por segundo. Para LEDs, frecuencias >100 Hz evitan el parpadeo visible al ojo humano. Para motores, 1-20 kHz es típico. Para servos, se usan pulsos de 50 Hz.
- En Arduino, analogWrite(pin, valor) genera PWM con valor de 0 (0% duty) a 255 (100% duty). La frecuencia por defecto es ~490 Hz en los pines 3, 9, 10, 11 y ~980 Hz en los pines 5 y 6.
- Servomotores: dispositivos que aceptan una señal PWM de 50 Hz donde el ancho del pulso (no el duty cycle) define la posición: 1 ms = 0 grados, 1.5 ms = 90 grados, 2 ms = 180 grados.
- La clase Servo de Arduino abstrae los detalles del protocolo PWM del servo: servo.write(grados) convierte los grados a los tiempos de pulso correctos automáticamente.

> **Nota:** Un LED conectado directamente a un pin PWM con resistencia limitadora (220 ohm para 5V) responde al duty cycle de forma casi lineal en brillo percibido. Sin embargo, la percepción humana del brillo es logarítmica, por lo que para una rampa suave de brillo se debe usar una curva gamma o un mapeo cuadrático del duty cycle.

MATERIALES Y HERRAMIENTAS

- Arduino UNO, Nano o ESP32 (cualquier plataforma con PWM)
- LED de cualquier color + resistencia 220 ohm
- Servomotor SG90 o MG90S (servo de 9g, muy económico)
- Potenciómetro de 10 kohm (para control manual del servo)
- Protoboard y jumpers
- Multímetro (opcional, para medir el voltaje promedio del PWM)

PROCEDIMIENTO PASO A PASO

1. Armar el circuito: LED con resistencia 220 ohm en pin 9 (PWM). Servomotor: VCC (rojo) a 5V, GND (negro/café) a GND, señal (naranja/amarillo) a pin 10. Potenciómetro: extremos a 5V y GND, wiper (cursor) a pin A0.
2. Cargar el sketch del laboratorio. Abrir el Monitor Serial a 115200 bps.
3. Observar la rampa de brillo del LED: debe ir de apagado a plena intensidad y volver suavemente (efecto 'respiración').
4. Al mismo tiempo, el servo debería moverse de 0 a 180 grados y regresar sincronizado con el LED.
5. Girar el potenciómetro lentamente. El servo debería seguir la posición del potenciómetro. El LED debería mostrar el brillo proporcional.
6. Usar el Monitor Serial para enviar comandos: 'D:128' para poner el LED al 50% de brillo, 'S:90' para mover el servo a 90 grados, 'AUTO' para iniciar la demo automática.
7. Con el multímetro en modo VDC, medir el voltaje en el pin del LED mientras cambia el duty cycle. Verificar que el voltaje promedio cambia proporcionalmente.
8. Probar con el potenciómetro al máximo, al centro y al mínimo. Medir el voltaje en A0 para cada posición y comparar con la lectura del ADC reportada por el serial.

Código de ejemplo:

```
// lab07_pwm_servo_dimmer.ino
#include <Servo.h>

const int PIN_LED   = 9;    // pin PWM para LED
const int PIN_SERVO = 10;   // pin PWM para servo
const int PIN_POT   = A0;   // entrada analogica del potenciometro

Servo miServo;
bool modoAuto = true;
int brightnessTarget = 0;
int servoTarget = 90;
String bufferCmd = "";

void setup() {
    Serial.begin(115200);
    pinMode(PIN_LED, OUTPUT);
    miServo.attach(PIN_SERVO);
    miServo.write(90);  // posicion inicial
    Serial.println("Lab 07 - PWM + Servo + ADC");
    Serial.println("Comandos: D:0-255 | S:0-180 | AUTO | POT");
}

void loop() {
    // Leer comandos seriales
    while (Serial.available()) {
        char c = Serial.read();
        if (c == '\n') { procesarComando(bufferCmd); bufferCmd = ""; }
```

```
        else if (c != '\r') bufferCmd += c;
    }

    if (modoAuto) {
        // Demo: rampa de brillo y movimiento de servo
        static int val = 0;
        static int dir = 1;
        static unsigned long ultimo = 0;
        if (millis() - ultimo > 10) {
            ultimo = millis();
            val += dir * 2;
            if (val >= 255) { val = 255; dir = -1; }
            if (val <= 0)   { val = 0;   dir =  1; }
            analogWrite(PIN_LED, val);
            int grados = map(val, 0, 255, 0, 180);
            miServo.write(grados);
        }
    } else {
        // Modo manual: potenciometro controla servo y LED
        int lecturaPot = analogRead(PIN_POT);  // 0-1023
        int grados = map(lecturaPot, 0, 1023, 0, 180);
        int brillo  = map(lecturaPot, 0, 1023, 0, 255);
        miServo.write(grados);
        analogWrite(PIN_LED, brillo);
        static unsigned long ultimoReport = 0;
        if (millis() - ultimoReport > 500) {
            ultimoReport = millis();
            float voltaje = lecturaPot * (5.0 / 1023.0);
            Serial.print("ADC="); Serial.print(lecturaPot);
            Serial.print(" V="); Serial.print(voltaje, 2);
            Serial.print(" Servo="); Serial.print(grados);
            Serial.print("deg  LED="); Serial.println(brillo);
        }
    }
}

void procesarComando(String cmd) {
    cmd.trim();
    if (cmd == "AUTO")  { modoAuto = true;  Serial.println("[OK] Modo
automatico"); }
    else if (cmd == "POT") { modoAuto = false; Serial.println("[OK] Modo
potenciometro"); }
    else if (cmd.startsWith("D:")) {
        int v = cmd.substring(2).toInt();
        v = constrain(v, 0, 255);
        analogWrite(PIN_LED, v);
        modoAuto = false;
        Serial.print("[OK] LED duty="); Serial.println(v);
    }
    else if (cmd.startsWith("S:")) {
        int g = cmd.substring(2).toInt();
        g = constrain(g, 0, 180);
        miServo.write(g);
        modoAuto = false;
```

```
        Serial.print("[OK] Servo="); Serial.print(g); Serial.println(" grados");
    }
}
```

PREGUNTAS DE REFLEXION Y REPORTE

- ¿Cuántos niveles de brillo distintos puede generar analogWrite() en Arduino UNO? ¿Cuántos bits tiene el DAC implícito del PWM?
- Si el servo recibe un pulso de 1.5 ms cada 20 ms (50 Hz), ¿cuál es el duty cycle en porcentaje? ¿Y para un pulso de 1 ms?
- ¿Por qué el LED parece aumentar de brillo de forma no lineal cuando el duty cycle va de 0 a 255 de manera lineal? ¿Qué cambio en el código produciría una rampa de brillo más uniforme perceptualmente?
- El ADC del ATmega328P es de 10 bits. ¿Cuántos niveles distintos puede diferenciar? Si el rango es de 0 a 5V, ¿cuál es la resolución en mV por nivel?
- ¿Como modificarías el sketch para que al recibir el comando 'SCAN', el servo haga un barrido de 0 a 180 y de regreso, mientras reporta la posición cada 10 grados vía serial?

LABORATORIO 08 | NIVEL: INTERMEDIO

SPI en Acción: Comunicación con Pantalla TFT ILI9341

Unidad: Unidad 2 - Microcontroladores | Duracion: 120 minutos | Temas: 2.3, 2.5

OBJETIVO

Implementar comunicación SPI entre un microcontrolador y una pantalla TFT de 2.4 pulgadas, dibujando formas geométricas, texto y datos de sensores en tiempo real para comprender el protocolo SPI en una aplicación real.

FUNDAMENTO TEORICO

SPI (Serial Peripheral Interface) es un protocolo síncrono full-duplex de alta velocidad utilizado ampliamente para pantallas, memorias flash, tarjetas SD y ADCs de precisión. Sus 4 señales son:

- SCK (Serial Clock): reloj generado por el master. Sincroniza cada bit de datos. Velocidades típicas: 1-40 MHz.
- MOSI (Master Out Slave In): datos del MCU hacia el dispositivo esclavo (pantalla).
- MISO (Master In Slave Out): datos del esclavo hacia el MCU. Muchas pantallas no lo usan (solo escritura).
- CS/DC: Chip Select activa el dispositivo; Data/Command distingue si el byte es un comando de control o un dato de pixel.

La pantalla ILI9341 es un controlador de pantalla TFT de 240x320 pixeles a color de 16 bits (RGB565). Puede recibir datos SPI hasta 40 MHz, permitiendo actualizar la pantalla completa varias veces por segundo.

- Formato de color RGB565: 5 bits rojo, 6 bits verde, 5 bits azul. Un pixel = 2 bytes. Pantalla completa = 240x320x2 = 153,600 bytes de datos.
- La biblioteca Adafruit GFX proporciona funciones de alto nivel: drawLine(), fillRect(), drawCircle(), print() que abstraen los comandos de bajo nivel del ILI9341.

> **Nota:** Una pantalla TFT consume típicamente 50-80 mA. No conectar directamente a batería de 3.7V sin regulador. El pin LED (backlight) acepta PWM para controlar el brillo.

MATERIALES Y HERRAMIENTAS

- ESP32 DevKit o Arduino UNO/Mega (ESP32 recomendado por mayor velocidad SPI)
- Pantalla TFT 2.4 pulgadas ILI9341 (240x320) con interfaz SPI
- Jumpers macho-hembra (8 piezas mínimo)
- Sensor BMP280 o DHT22 (para mostrar datos en pantalla)
- Arduino IDE 2.x con bibliotecas: Adafruit ILI9341, Adafruit GFX instaladas

PROCEDIMIENTO PASO A PASO

1. Conexión de la pantalla ILI9341 al ESP32: VCC->3.3V, GND->GND, CS->GPIO5, RESET->GPIO22, DC->GPIO21, MOSI->GPIO23 (SPI MOSI), SCK->GPIO18 (SPI CLK), LED->3.3V (o pin PWM para control de brillo).
2. En Arduino UNO: CS->pin 10, RESET->pin 8, DC->pin 9, MOSI->pin 11, SCK->pin 13, LED->5V via resistencia 100 ohm.
3. Instalar las bibliotecas Adafruit ILI9341 y Adafruit GFX Library via Administrador de Bibliotecas.
4. Cargar el ejemplo de la biblioteca: Archivo > Ejemplos > Adafruit ILI9341 > graphicstest. Verificar que la pantalla muestra la demo de figuras geométricas.
5. Cargar el sketch del laboratorio que dibuja las formas básicas y texto.
6. Modificar el sketch para mostrar el valor de temperatura del sensor en la pantalla, actualizado cada segundo.
7. Experimentar con los colores: ILI9341_RED, ILI9341_GREEN, ILI9341_BLUE, ILI9341_WHITE, ILI9341_YELLOW. Dibujar la bandera de Mexico simplificada.
8. Medir con el cronometro cuanto tarda en actualizarse la pantalla completa con fillScreen() a diferente velocidad SPI (ajustando el parámetro en el constructor).

Código de ejemplo:

```
// lab08_spi_tft_ili9341.ino (ESP32)
#include <Adafruit_GFX.h>
#include <Adafruit_ILI9341.h>
#include <SPI.h>

// Pines SPI para ESP32
#define TFT_CS   5
#define TFT_DC  21
#define TFT_RST 22

Adafruit_ILI9341 tft = Adafruit_ILI9341(TFT_CS, TFT_DC, TFT_RST);

// Colores custom RGB565
#define NARANJA 0xFD20
#define GRIS    0x8410

void setup() {
    Serial.begin(115200);
    tft.begin();
    tft.setRotation(1);  // landscape
    tft.fillScreen(ILI9341_BLACK);
    // Pantalla de bienvenida
    tft.setTextColor(ILI9341_CYAN);
    tft.setTextSize(2);
    tft.setCursor(20, 20);
    tft.println("Lab 08 - SPI TFT");
    tft.setTextColor(ILI9341_WHITE);
    tft.setTextSize(1);
```

```
    tft.setCursor(20, 50);
    tft.println("ILI9341 240x320 @ 40MHz SPI");
    delay(1500);
    tft.fillScreen(ILI9341_BLACK);
    dibujarPanel();
}

float temperatura = 24.5;  // Reemplazar con lectura real del sensor
float humedad = 58.3;
int frameCount = 0;

void loop() {
    frameCount++;
    // Simular variacion del sensor (reemplazar con lectura I2C real)
    temperatura = 24.5 + sin(frameCount * 0.05) * 3.0;
    humedad     = 58.0 + cos(frameCount * 0.03) * 5.0;
    actualizarDatos(temperatura, humedad);
    delay(500);
}

void dibujarPanel() {
    // Barra superior
    tft.fillRect(0, 0, 320, 30, P.midBlue);  // usar valor numerico 0x2471
    tft.fillRect(0, 0, 320, 30, 0x2471);
    tft.setTextColor(ILI9341_WHITE); tft.setTextSize(1);
    tft.setCursor(10, 10); tft.print("ESTACION AMBIENTAL  |  ESP32 +
ILI9341");
    // Marco temperatura
    tft.drawRoundRect(10, 40, 140, 100, 8, ILI9341_CYAN);
    tft.setTextColor(ILI9341_CYAN); tft.setTextSize(1);
    tft.setCursor(20, 55); tft.print("TEMPERATURA");
    // Marco humedad
    tft.drawRoundRect(165, 40, 140, 100, 8, ILI9341_GREEN);
    tft.setTextColor(ILI9341_GREEN); tft.setTextSize(1);
    tft.setCursor(175, 55); tft.print("HUMEDAD");
    // Grafica historica (area inferior)
    tft.drawRect(10, 155, 300, 75, GRIS);
    tft.setTextColor(GRIS); tft.setTextSize(1);
    tft.setCursor(15, 160); tft.print("Historial temp.");
}

void actualizarDatos(float temp, float hum) {
    // Actualizar temperatura (borrar area previa y redibujar)
    tft.fillRect(15, 70, 125, 55, ILI9341_BLACK);
    tft.setTextColor(ILI9341_WHITE); tft.setTextSize(3);
    tft.setCursor(20, 78);
    tft.print(temp, 1); tft.print(" C");
    // Actualizar humedad
    tft.fillRect(170, 70, 125, 55, ILI9341_BLACK);
    tft.setTextColor(ILI9341_WHITE); tft.setTextSize(3);
    tft.setCursor(175, 78);
    tft.print(hum, 1); tft.print(" %");
}
```

PREGUNTAS DE REFLEXION Y REPORTE

- ¿Cuántos bytes de datos se necesitan para actualizar un solo pixel en la pantalla ILI9341? ¿Y para actualizar toda la pantalla (240x320)?
- A 40 MHz de SPI, ¿cuánto tiempo mínimo (teórico) se necesita para enviar todos los datos de un frame completo? ¿Por qué en la práctica es mayor?
- ¿Por qué se usa la función fillRect() para borrar el área del número antes de redibujar el nuevo valor, en lugar de usar fillScreen() completo?
- Compara la velocidad de actualización de la pantalla cuando el ESP32 opera el SPI a 10 MHz vs 40 MHz. ¿Qué diferencia se observa visualmente?
- ¿Cuáles son las ventajas de usar SPI sobre I2C para una pantalla grafica de alta resolución? ¿Existiría alguna ventaja de usar I2C para este caso?

LABORATORIO 09 | NIVEL: INTERMEDIO

ESP32 como Servidor Web: IoT con WiFi Integrado

Unidad: Unidad 2 - Microcontroladores | Duración: 150 minutos | Temas: 2.3, 2.4, 2.5

OBJETIVO

Configurar el ESP32 como punto de acceso WiFi y servidor web HTTP embebido, publicando datos de sensores en tiempo real accesibles desde cualquier navegador en la red local.

FUNDAMENTO TEORICO

El ESP32 integra en un único SoC un procesador dual-core Xtensa LX6 y un módulo WiFi 802.11 b/g/n completo. Esto permite implementar servidores web sin hardware adicional. Las capas involucradas son:

- WiFi Stack: el ESP32 puede operar como Station (STA, se conecta a un router existente), Access Point (AP, crea su propia red WiFi), o en modo dual STA+AP simultáneamente.
- TCP/IP Stack: LwIP (Lightweight IP) es el stack de red integrado. Maneja ARP, DHCP, TCP, UDP e IP automáticamente.
- Servidor HTTP: escucha en el puerto 80 conexiones TCP. Cuando un navegador conecta, el ESP32 responde con HTML/CSS/JavaScript. La clase WiFiServer y WiFiClient del framework Arduino abstraen esta funcionalidad.
- Protocolo HTTP: basado en texto. El navegador envía 'GET / HTTP/1.1', el servidor responde con 'HTTP/1.1 200 OK' seguido de los headers y el cuerpo HTML.

Para datos en tiempo real sin recargar la página se puede usar JavaScript con fetch() periódico o WebSockets. En este laboratorio se implementa actualización automática vía meta refresh y también un endpoint JSON consultable con fetch().

> **Nota:** El ESP32 tiene 520 KB de SRAM y hasta 4 MB de Flash. El servidor web embebido puede manejar 2-4 clientes concurrentes sin problema. Para mayor concurrencia se usa FreeRTOS con tareas independientes por conexión.

MATERIALES Y HERRAMIENTAS

- ESP32 DevKit v1 o v2 (cualquier variante con WiFi)
- Sensor DHT22 o BMP280 (para tener datos reales que mostrar)
- Resistencia 10 kohm (pull-up para DHT22)
- Cable USB y PC con WiFi
- Arduino IDE con soporte ESP32 instalado (esp32 by Espressif en Board Manager)
- Biblioteca DHT sensor library (Adafruit) instalada
- Navegador web en la PC o smartphone

PROCEDIMIENTO PASO A PASO

1. Instalar el soporte ESP32 en Arduino IDE: en Preferencias, agregar 'https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json' en URLs adicionales. Luego en Board Manager, buscar 'esp32' e instalar.
2. Conectar el DHT22: pin datos al GPIO4 del ESP32 con resistencia 10 kohm pull-up a 3.3V. VCC a 3.3V, GND a GND.
3. Modificar el sketch: reemplazar 'TU_SSID' y 'TU_PASSWORD' con las credenciales de tu red WiFi local.
4. Cargar el sketch. Abrir el Monitor Serial a 115200 bps.
5. Esperar que el ESP32 se conecte al WiFi. La IP asignada se mostrará en el monitor serial (ej. '192.168.1.105').
6. En el navegador de la PC (conectada a la misma red), navegar a 'http://192.168.1.105' (usar la IP que reporto el serial).
7. Verificar que la página muestra temperatura y humedad actualizadas. La página se recarga automáticamente cada 3 segundos.
8. Desde el navegador, acceder a 'http://[IP]/api/data' para ver el endpoint JSON crudo.
9. Abrir el mismo URL desde el smartphone (en la misma red WiFi) para verificar que funciona en cualquier dispositivo.

Código de ejemplo:

```
// lab09_esp32_webserver.ino
#include <WiFi.h>
#include <DHT.h>

// --- Configuracion WiFi ---
const char* ssid     = "TU_SSID";
const char* password = "TU_PASSWORD";

// --- Sensor DHT22 ---
const int DHT_PIN  = 4;
#define DHTTYPE DHT22
DHT dht(DHT_PIN, DHTTYPE);

WiFiServer server(80);
float temperatura = 0, humedad = 0;
unsigned long ultimaLectura = 0;

void setup() {
    Serial.begin(115200);
    dht.begin();
    Serial.print("Conectando a: "); Serial.println(ssid);
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500); Serial.print(".");
    }
    Serial.println("\n[WiFi] Conectado!");
    Serial.print("[WiFi] IP: "); Serial.println(WiFi.localIP());
```

```
    server.begin();
    Serial.println("[HTTP] Servidor iniciado en puerto 80");
}

void loop() {
    // Actualizar sensor cada 3 segundos
    if (millis() - ultimaLectura > 3000) {
        ultimaLectura = millis();
        float t = dht.readTemperature();
        float h = dht.readHumidity();
        if (!isnan(t) && !isnan(h)) { temperatura = t; humedad = h; }
    }
    // Atender clientes HTTP
    WiFiClient client = server.available();
    if (client) {
        String request = "";
        while (client.available()) {
            char c = client.read();
            request += c;
            if (request.endsWith("\r\n\r\n")) break;
        }
        // Ruteo de endpoints
        if (request.indexOf("GET /api/data") >= 0) {
            enviarJSON(client);
        } else {
            enviarHTML(client);
        }
        client.stop();
    }
}

void enviarJSON(WiFiClient& client) {
    client.println("HTTP/1.1 200 OK");
    client.println("Content-Type: application/json");
    client.println("Access-Control-Allow-Origin: *");
    client.println();
    client.print('{');
    client.print("\"temperatura\":"); client.print(temperatura, 1);
client.print(',');
    client.print("\"humedad\":");     client.print(humedad, 1);
client.print(',');
    client.print("\"uptime_s\":");   client.print(millis()/1000);
client.print(',');
    client.print("\"rssi_dbm\":");   client.print(WiFi.RSSI());
    client.println('}');
}

void enviarHTML(WiFiClient& client) {
    client.println("HTTP/1.1 200 OK");
    client.println("Content-Type: text/html; charset=utf-8");
    client.println();
    client.println("<!DOCTYPE html><html><head>");
    client.println("<meta charset='utf-8'>");
    client.println("<meta name='viewport' content='width=device-width,
initial-scale=1'>");
```

```
    client.println("<title>ESP32 Sensor</title>");
    client.println("<style>body{font-
family:Arial;background:#1a2f4a;color:#fff;text-align:center;}");
    client.println(".card{background:#2471a3;border-
radius:12px;padding:20px;margin:10px;display:inline-block;}");
    client.println(".val{font-size:48px;font-weight:bold;color:#1abc9c;}");
    client.println(".lbl{font-size:14px;color:#aaa;}</style>");
    client.println("<script>");
    client.println("setInterval(async()=>{");
    client.println("  const r=await fetch('/api/data'); const d=await
r.json();");
    client.println("
document.getElementById('t').textContent=d.temperatura.toFixed(1)+'C';");
    client.println("
document.getElementById('h').textContent=d.humedad.toFixed(1)+'%';");
    client.println("  document.getElementById('up').textContent='Uptime:
'+d.uptime_s+'s | RSSI: '+d.rssi_dbm+'dBm';");
    client.println("},3000);</script></head><body>");
    client.println("<h2>Estacion Ambiental ESP32</h2>");
    client.println("<div class='card'><div class='lbl'>TEMPERATURA</div>");
    client.print("<div class='val' id='t'>"); client.print(temperatura,1);
client.println("C</div></div>");
    client.println("<div class='card'><div class='lbl'>HUMEDAD</div>");
    client.print("<div class='val' id='h'>"); client.print(humedad,1);
client.println("%</div></div>");
    client.println("<p id='up' style='color:#888'></p>");
    client.println("</body></html>");
}
```

PREGUNTAS DE REFLEXION Y REPORTE

- ¿Qué significa el código de respuesta HTTP '200 OK'? ¿Qué otros códigos HTTP conoces y cuando se usarían en un servidor embebido?
- ¿Por qué se usa fetch() con JavaScript en lugar del meta refresh para actualizar los datos? ¿Cuál es la ventaja para el usuario y para el servidor?
- ¿Como se podría extender el servidor para que el cliente pueda enviar comandos al ESP32 desde el navegador (ej. encender un LED vía botón en la pagina HTML)?
- Si el ESP32 tiene 520 KB de SRAM y el programa del servidor usa ~120 KB, ¿cuanta memoria queda para el heap del servidor y los buffers de conexión?
- ¿Qué diferencia hay entre que el ESP32 opere como Station (STA) y como Access Point (AP)? ¿En qué escenario practico usarías cada modo?

LABORATORIO 10 | NIVEL: INTERMEDIO

Benchmark y Comparativa de Arquitecturas x86 vs ARM

Unidad: Unidad 3 - Equipos Empresariales | Duración: 120 minutos | Temas: 3.2, 3.3

OBJETIVO

Ejecutar benchmarks estandarizados en procesadores x86-64 y ARM disponibles, analizar los resultados de rendimiento y eficiencia energética, y elaborar una comparativa técnica fundamentada.

FUNDAMENTO TEORICO

La evaluación objetiva de procesadores requiere benchmarks estandarizados que midan diferentes dimensiones del rendimiento:

- Single-Core: mide la velocidad de un único hilo de ejecución. Crítico para aplicaciones que no pueden paralelizarse (hojas de cálculo, muchos videojuegos, ciertos servicios web).
- Multi-Core: mide el rendimiento usando todos los núcleos disponibles. Relevante para compilación de código, encoding de video, servidores de alta concurrencia, renderizado 3D.
- Geekbench 6 Compute: evalúa el rendimiento de la GPU y NPU para operaciones de punto flotante y aprendizaje automático.
- Rendimiento por vatio (Performance/Watt): la métrica de eficiencia energética más importante para dispositivos móviles, servidores en la nube y cualquier entorno donde el costo de electricidad o la autonomía de batería sean críticos.
- IPC (Instructions Per Clock): caracteriza la microarquitectura independientemente de la frecuencia. Procesadores ARM modernos (Cortex-X4, Apple Everest) tienen IPC comparable o superior a x86.

La Ley de Dennard (scaling de potencia con la frecuencia) se rompió alrededor de 2005, lo que causo el fin de la guerra de GHz y el inicio de la era multicore. Hoy, la eficiencia energética es tan importante como el rendimiento bruto.

MATERIALES Y HERRAMIENTAS

- PC o laptop con procesador x86-64 (Intel Core o AMD Ryzen)
- Smartphone Android o iPhone para benchmark ARM (o Mac con Apple Silicon si disponible)
- Geekbench 6 gratuito: pc desde geekbench.com, móvil desde la app store
- HWiNFO64 o similar para medir consumo durante el benchmark en PC
- Acceso a internet para consultar resultados en browser.geekbench.com
- Hoja de cálculo para análisis de resultados

PROCEDIMIENTO PASO A PASO

1. En la PC: descargar e instalar Geekbench 6 desde geekbench.com. Cerrar todas las aplicaciones en segundo plano. Conectar el cargador si es laptop.
2. Abrir HWiNFO64, ir a Sensors y localizar el valor 'CPU Package Power' o equivalente. Anotar el consumo en reposo (idle) antes de iniciar el benchmark.
3. Ejecutar Geekbench 6 en la PC. Mientras corre, observar el consumo de CPU en HWiNFO64 y anotar el consumo máximo durante la prueba multi-core.
4. Anotar los puntajes Single-Core y Multi-Core. Calcular: rendimiento por vatio = puntaje Multi-Core / consumo en Watts durante el test.
5. En el smartphone: instalar Geekbench 6 desde la tienda de apps. Asegurarse de que la batería este al 50-100% y el teléfono no este sobrecalentado.
6. Ejecutar Geekbench 6 en el smartphone. Anotar los puntajes Single-Core y Multi-Core.
7. Buscar los resultados de ambos procesadores en browser.geekbench.com para comparar con la media global de dispositivos similares.
8. Construir una tabla comparativa con: CPU, arquitectura, proceso de fab. (nm), núcleos, frecuencia, TDP, SC score, MC score, y rendimiento/vatio calculado.
9. Calcular que porcentaje del máximo Multi-Core se aprovecha en el test Single-Core. Comparar este ratio entre x86 y ARM.

Código de ejemplo:

```
// Python: script para analisis de datos de benchmarks
# Ejecutar en PC con Python 3 instalado

datos_procesadores = [
    {'nombre': 'Intel Core i7-1280P',  'arq': 'x86-64', 'nm': 10, 'cores': 14,
     'freq_ghz': 1.8, 'tdp_w': 28,  'gb6_sc': 2200, 'gb6_mc': 11500},
    {'nombre': 'AMD Ryzen 7 7745HX',   'arq': 'x86-64', 'nm': 5,  'cores': 8,
     'freq_ghz': 3.6, 'tdp_w': 45,  'gb6_sc': 2600, 'gb6_mc': 15000},
    {'nombre': 'Apple M3',             'arq': 'ARM64',  'nm': 3,  'cores': 8,
     'freq_ghz': 4.0, 'tdp_w': 22,  'gb6_sc': 3100, 'gb6_mc': 14800},
    {'nombre': 'Snapdragon 8 Gen 3',   'arq': 'ARM64',  'nm': 4,  'cores': 8,
     'freq_ghz': 3.3, 'tdp_w': 8,   'gb6_sc': 2400, 'gb6_mc': 8800},
    # Agregar datos reales medidos en el laboratorio:
    # {'nombre': 'Tu CPU', 'arq': 'x86-64', 'nm': ?, 'cores': ?, ...
]

print(f"{'Procesador':<30} {'Arq':<8} {'nm':<5} {'TDP':>6} {'SC':>7} {'MC':>8}
{'MC/W':>8}")
print('-' * 80)
for d in datos_procesadores:
    mc_por_w = d['gb6_mc'] / d['tdp_w']
    print(f"{d['nombre']:<30} {d['arq']:<8} {d['nm']:<5} {d['tdp_w']:>5}W
{d['gb6_sc']:>7} {d['gb6_mc']:>8} {mc_por_w:>7.0f}")

# Encontrar el mas eficiente
mejor = max(datos_procesadores, key=lambda x: x['gb6_mc']/x['tdp_w'])
```

```
print(f"\nMas eficiente (MC/W): {mejor['nombre']} -
{mejor['gb6_mc']/mejor['tdp_w']:.0f} pts/W")
```

PREGUNTAS DE REFLEXION Y REPORTE

- ¿Qué procesador obtuvo mejor rendimiento Single-Core? ¿Y Multi-Core? ¿Son el mismo? ¿Por qué podrían ser distintos?
- ¿Cuál es el ratio de rendimiento por vatio entre el procesador x86 y el ARM analizado? ¿Como se explica esta diferencia desde el punto de vista arquitectónico?
- Si tuvieras que elegir entre el procesador x86 y el ARM para: (a) un servidor de base de datos con alta concurrencia, (b) un laptop para estudiante con batería máxima, ¿cuál elegirías para cada caso? Justifica con los números obtenidos.
- ¿Por qué los procesadores ARM modernos (Apple M-series) pueden igualar o superar a x86 en Single-Core a pesar de que ARM es 'Reduced Instruction Set'? ¿Qué otras mejoras micro arquitectónicas explican esto?
- Calcula el costo estimado de electricidad anual de un servidor con el procesador x86 analizado operando al 60% de carga 24/7, con un costo de $2.50/kWh (precio típico en Mexico en 2024).

LABORATORIO 11 | NIVEL: INTERMEDIO

Diagnóstico de Hardware con SMART y Herramientas del Sistema

Unidad: Unidad 3 - Equipos Empresariales | Duración: 90 minutos | Temas: 3.5

OBJETIVO

Aplicar herramientas de diagnóstico de hardware para evaluar el estado de salud de los componentes de una PC real, interpretar los datos SMART de los discos e identificar posibles problemas o riesgos.

FUNDAMENTO TEORICO

El diagnostico preventivo de hardware es una disciplina esencial en la administración de TI. Las herramientas de diagnóstico leen datos de los sensores integrados en los componentes:

- SMART (Self-Monitoring, Analysis and Reporting Technology): sistema integrado en HDD y SSD que registra continuamente métricas de salud. Los atributos críticos son: Reallocated Sector Count (sectores malos reasignados), Current Pending Sectors (sectores pendientes de reasignación), Uncorrectable Sector Count (errores sin correccion), y Power On Hours.
- Temperatura: cada componente tiene un sensor interno. Los umbrales críticos son: CPU <90 grados C bajo carga, HDD <55 grados C, SSD NVMe <80 grados C. Temperaturas persistentemente altas aceleran la degradación.
- Voltajes del sistema: los rieles principales son +12V (CPU, GPU, motores de disco), +5V (circuitos lógicos, USB), +3.3V (RAM, lógica de bajo voltaje). Tolerancia estándar: +/-5%.
- Errores ECC (en RAM de servidor): la RAM ECC registra cuantos errores de 1 bit ha corregido. Muchos errores corregidos indican que el módulo se aproxima al fallo total.

La filosofía del mantenimiento predictivo es: los componentes avisan antes de fallar. Si monitorizamos los datos correctos, podemos reemplazar un disco o limpiar un disipador antes de que cause una pérdida de datos o una interrupción del servicio.

> **Nota:** En entornos empresariales, herramientas como Nagios, Zabbix, Prometheus + Grafana monitorizan estos parámetros de forma continua en cientos o miles de servidores, generando alertas automáticas cuando algún valor supera un umbral.

MATERIALES Y HERRAMIENTAS

- PC de escritorio o laptop (preferiblemente una que tenga HDD o SSD de más de 2 años de uso)
- CrystalDiskInfo 9.x (Windows, gratuito en crystalmark.info)
- HWiNFO64 (Windows, gratuito en hwinfo.com)
- En Linux: smartmontools (sudo apt install smartmontools), lm-sensors

- Cinebench 2024 o Prime95 para el test de estrés (Windows)
- Papel y lápiz para anotar lecturas de temperatura en distintos momentos

PROCEDIMIENTO PASO A PASO

1. Instalar CrystalDiskInfo y HWiNFO64. Abrir CrystalDiskInfo: anotar para cada disco presente: modelo, número de serie, interfaz (SATA/NVMe/USB), temperatura actual, Power On Hours, y el estado general (Bueno/Precaucion/Malo).
2. Identificar los valores SMART críticos: buscar los atributos ID 05 (Reallocated Sector Count), ID C5 (Current Pending Sectors), ID C6 (Uncorrectable Sector Count). Si alguno tiene valor RAW no-cero, es una señal de advertencia seria.
3. Para SSD: buscar el atributo Wear Leveling Count o Percentage Used/Remaining. Anotar el valor. Un SSD con menos del 10% de vida restante deberia reemplazarse pronto.
4. En Linux: ejecutar 'sudo smartctl -a /dev/sda' (o nvme0n1 para NVMe) y anotar los mismos atributos.
5. Abrir HWiNFO64 > Sensors. Anotar las temperaturas base (sin carga) de: CPU, GPU (si tiene), y todos los discos. Esperar 2 minutos.
6. Ejecutar Cinebench 2024 Multi-Core (o Prime95 por 5 minutos) mientras HWiNFO64 está abierto. Anotar la temperatura máxima alcanzada por el CPU y si la frecuencia se reduce (throttling).
7. Revisar los voltajes en HWiNFO64: documentar los valores de +12V, +5V y +3.3V. Compararlos con los valores nominales.
8. Revisar el Event Viewer de Windows (eventvwr.msc) > Registros de Windows > Sistema. Filtrar por tipo 'Critico' y 'Error' del último mes. Identificar y documentar los 3 eventos más recurrentes.
9. Elaborar un informe de diagnóstico con: estado de cada componente, valores medidos, identificación de riesgos y recomendaciones de acción.

Código de ejemplo:

```
# Linux: diagnostico completo de disco con smartctl
sudo smartctl -a /dev/sda                # SMART completo
sudo smartctl -t short /dev/sda          # iniciar test corto (tarda ~2 min)
sudo smartctl -H /dev/sda                # solo estado de salud

# Ver temperatura del disco en tiempo real
sudo hddtemp /dev/sda

# Informacion de NVMe
sudo smartctl -a /dev/nvme0
sudo nvme smart-log /dev/nvme0n1

# Monitoreo de sensores de temperatura CPU/MB
sudo apt install lm-sensors
sudo sensors-detect    # detectar sensores (responder 'yes' a todo)
sensors                # ver temperaturas actuales
watch -n 2 sensors     # monitoreo cada 2 segundos
```

```
# Resumen completo del hardware del sistema
sudo inxi -Fxz

# Ver eventos de error del kernel relacionados con hardware
sudo dmesg | grep -i -E 'error|warn|fail|critical' | tail -30

# Ver errores de memoria (si hay ECC)
sudo dmidecode --type 16    # tipo de memoria
sudo edac-util -s 0 2>/dev/null || echo 'ECC no disponible/habilitado'
```

PREGUNTAS DE REFLEXION Y REPORTE

- ¿Cuál es el estado SMART de los discos analizados? Si algún atributo critico tiene valor RAW no-cero, ¿qué acción inmediata debería tomarse?
- ¿Cuánto tiempo llevan encendidos los discos (Power On Hours)? Si un HDD ha operado 40,000 horas (~4.5 años), ¿debería considerarse su reemplazo preventivo? Justifica con datos de confiabilidad de discos.
- ¿Cuánta diferencia de temperatura hay entre el CPU en reposo y bajo carga máxima? ¿Se observo throttling (reducción de frecuencia por temperatura alta)? ¿Que indica esto sobre el sistema de refrigeración?
- ¿Los voltajes del sistema están dentro del rango de tolerancia del 5%? Si el +12V mide 11.4V, ¿está dentro del rango aceptable? ¿Y si mide 11V?
- Diseña un plan de mantenimiento preventivo para los próximos 12 meses para el equipo analizado, basado en los hallazgos de este laboratorio. Incluye frecuencia de cada acción y el umbral que requeriría acción correctiva urgente.

LABORATORIO 12 | NIVEL: INTERMEDIO

Virtualización y Análisis de Overhead: Nativo vs VM vs Contenedor

Unidad: Unidad 3 - Equipos Empresariales | Duración: 150 minutos | Temas: 3.2, 3.3, 3.4

OBJETIVO

Comparar el rendimiento de una aplicación ejecutada en bare metal (nativo), dentro de una máquina virtual y dentro de un contenedor Docker, cuantificando el overhead de cada capa de virtualización.

FUNDAMENTO TEORICO

La virtualización es la tecnología base de la computación en la nube moderna. Permite ejecutar múltiples sistemas operativos aislados en el mismo hardware físico. Existen tres modelos principales:

- Bare metal (nativo): el sistema operativo accede directamente al hardware. Máximo rendimiento, sin overhead. Referencia para comparar.
- Máquina Virtual (VM): el hipervisor (VMware ESXi, Hyper-V, KVM, VirtualBox) crea una abstracción del hardware físico. Cada VM tiene su propio kernel. Overhead típico: 3-15% en CPU y hasta 20-30% en I/O de disco.
- Contenedor (Docker/Podman): comparte el kernel del host. Solo aísla el espacio de procesos, red y sistema de archivos. Overhead muy bajo: 1-3% en CPU, más latencia en I/O de red. Inicio en milisegundos vs minutos de VM.
- Hipervisor Tipo 1 (Bare Metal): se instala directamente sobre el hardware sin SO intermedio. Ejemplos: VMware ESXi, Microsoft Hyper-V Server, KVM (en Linux). Menor overhead que Tipo 2.
- Hipervisor Tipo 2 (Hosted): se instala sobre un SO existente como una aplicación. Ejemplos: VirtualBox, VMware Workstation. Mayor overhead pero más fácil de usar en estaciones de trabajo.

Para el desarrollador y el administrador de sistemas, la pregunta practica es: ¿cuánto rendimiento pierdo al virtualizar? Este laboratorio cuantifica ese overhead con benchmarks reproducibles.

MATERIALES Y HERRAMIENTAS

- PC con Windows 10/11 o Linux con al menos 8 GB de RAM y 20 GB libres en disco
- VirtualBox 7.x instalado (virtualbox.org, gratuito)
- ISO de Ubuntu Server 24.04 LTS descargada
- Docker Desktop instalado (docker.com) o Docker Engine en Linux
- Python 3 instalado en el sistema nativo, en la VM y en el contenedor
- Sysbench instalado (sudo apt install sysbench) en Linux, o script Python equivalente

PROCEDIMIENTO PASO A PASO

1. Crear una VM en VirtualBox: Nueva VM, tipo Linux (Ubuntu 64-bit), 2 GB RAM, 1 CPU, disco de 20 GB. Instalar Ubuntu Server 24.04. Instalar sysbench: 'sudo apt install sysbench'.
2. BENCHMARK NATIVO (en el host): ejecutar las pruebas de CPU y memoria del script Python provisto. Anotar los resultados en la tabla.
3. BENCHMARK EN VM: iniciar la VM de Ubuntu Server. Ejecutar las mismas pruebas desde la terminal de la VM. Anotar los resultados.
4. Instalar Docker en el sistema host (si no esta instalado). Descargar la imagen: 'docker pull python:3.12-slim'.
5. BENCHMARK EN CONTENEDOR: ejecutar el script dentro de un contenedor Docker: 'docker run --rm -v $(pwd):/app python:3.12-slim python /app/benchmark.py'. Anotar resultados.
6. Calcular el overhead de la VM y del contenedor respecto al nativo: overhead% = ((tiempo_vm - tiempo_nativo) / tiempo_nativo) * 100.
7. Medir el tiempo de inicio de la VM (desde encendido hasta login funcional) y del contenedor (desde 'docker run' hasta ejecución del primer comando).
8. Comparar el consumo de RAM: VM tiene su asignación fija (2 GB), contenedor usa solo lo que la aplicación necesita. Verificar con 'free -h' en la VM y 'docker stats' para el contenedor.

Código de ejemplo:

```
# benchmark.py - Ejecutar en nativo, VM y contenedor Docker
import time
import math
import sys

def benchmark_cpu(iteraciones=5_000_000):
    """Prueba de CPU: operaciones matematicas intensivas"""
    inicio = time.perf_counter()
    resultado = 0.0
    for i in range(1, iteraciones + 1):
        resultado += math.sqrt(i) * math.log(i)
    elapsed = time.perf_counter() - inicio
    return elapsed, resultado

def benchmark_memoria(tamano_mb=256):
    """Prueba de memoria: asignacion y escritura de buffer grande"""
    inicio = time.perf_counter()
    buffer = bytearray(tamano_mb * 1024 * 1024)
    for i in range(0, len(buffer), 4096):
        buffer[i] = i % 256  # escribir cada pagina
    elapsed = time.perf_counter() - inicio
    return elapsed

def benchmark_disco(nombre='test_io.bin', tamano_mb=64):
    """Prueba de I/O: escritura y lectura de archivo"""
```

```
    datos = b'x' * (1024 * 1024)  # 1 MB de datos
    # Escritura
    t0 = time.perf_counter()
    with open(nombre, 'wb') as f:
        for _ in range(tamano_mb):
            f.write(datos)
    t_write = time.perf_counter() - t0
    # Lectura
    t0 = time.perf_counter()
    with open(nombre, 'rb') as f:
        while f.read(1024*1024): pass
    t_read = time.perf_counter() - t0
    import os; os.remove(nombre)
    return t_write, t_read

print(f'Entorno: {sys.platform} | Python {sys.version.split()[0]}')
print('='*55)

t_cpu, _ = benchmark_cpu()
print(f'CPU (5M iteraciones):     {t_cpu:.3f} s')

t_mem = benchmark_memoria()
print(f'Memoria (256 MB alloc):   {t_mem:.3f} s')

t_w, t_r = benchmark_disco()
print(f'Disco escritura (64MB):   {t_w:.3f} s  ({64/t_w:.1f} MB/s)')
print(f'Disco lectura  (64MB):    {t_r:.3f} s  ({64/t_r:.1f} MB/s)')
print('='*55)
```

PREGUNTAS DE REFLEXION Y REPORTE

- ¿Cuánto overhead de CPU introdujo la VM respecto al nativo? ¿Y el contenedor Docker? ¿Coincide con los valores típicos mencionados en el fundamento teórico?
- ¿En qué tipo de operación (CPU, memoria o disco) se observó mayor overhead de la VM? ¿Por qué el overhead de disco suele ser mayor que el de CPU en virtualización?
- ¿Cuánto tiempo tardo en iniciar la VM de Ubuntu Server vs el contenedor Docker? ¿Qué implicación tiene esta diferencia en un entorno de microservicios que necesita escalar rápidamente?
- Si una empresa necesita ejecutar 20 instancias aisladas de una aplicación web Python, ¿necesita 20 VMs de Ubuntu o podría usar 20 contenedores Docker? ¿Cuál sería la diferencia en consumo de RAM?
- ¿En qué escenario usarías una VM en lugar de un contenedor, a pesar de su mayor overhead? Da un ejemplo concreto donde el aislamiento completo de la VM sea necesario.

LABORATORIO INTEGRADOR | NIVEL INTERMEDIO-AVANZADO

Sistema de Monitoreo Inteligente de Infraestructura TI

Unidades 1, 2 y 3 | Duración: 4-5 horas (puede distribuirse en 2-3 sesiones) | Todos los temas

DESCRIPCION DEL PROYECTO

El laboratorio integrador conecta los tres ejes fundamentales del curso en un único proyecto funcional:

- Unidad 1 (Arquitectura Digital): comprender como la jerarquía de memoria, el pipeline y el multicore impactan en el rendimiento del sistema de monitoreo.
- Unidad 2 (Microcontroladores): usar un ESP32 como nodo sensor embebido que recopila datos del hardware y del ambiente, comunicándose vía WiFi y UART.
- Unidad 3 (Equipos Empresariales): desplegar un servidor de monitoreo en una PC/servidor y tomar decisiones de mantenimiento basadas en los datos recopilados.

El sistema resultante es una plataforma de monitoreo de infraestructura TI a escala de laboratorio, conceptualmente equivalente a sistemas empresariales como Zabbix, Nagios o Prometheus.

OBJETIVO GENERAL

Diseñar, construir y documentar un sistema distribuido de monitoreo de infraestructura TI que combine: (1) un nodo embebido ESP32 que recopila métricas del ambiente físico del servidor, (2) un servidor central que agrega las métricas del hardware de la PC con Python, y (3) un dashboard web accesible desde la red local que visualiza en tiempo real el estado de salud de la infraestructura.

ARQUITECTURA DEL SISTEMA

El sistema tiene tres capas funcionales:

- Capa 1 - Nodo Embebido (ESP32): mide temperatura ambiente, humedad y opcionalmente CO_2 del cuarto de servidores. Envía los datos vía HTTP POST al servidor central cada 30 segundos. Tiene un LED de alerta que parpadea si la temperatura supera el umbral configurado.
- Capa 2 - Servidor de Monitoreo (PC/Laptop con Python): ejecuta un servidor Flask que recibe los datos del ESP32, recopila métricas del hardware local (CPU, RAM, disco vía psutil), almacena historiales en SQLite, y expone una API REST + dashboard web.
- Capa 3 - Dashboard Web (navegador): pagina HTML/JavaScript que consume la API del servidor Flask y muestra graficas en tiempo real de temperatura, uso de CPU, RAM y almacenamiento. Auto actualiza cada 5 segundos.

MATERIALES DEL LABORATORIO

- ESP32 DevKit v1 o v2 con soporte WiFi
- Sensor DHT22 (temperatura y humedad ambiente)
- LED rojo + resistencia 220 ohm (alerta visual)
- Protoboard y jumpers
- PC o laptop con Python 3.10+ instalado
- Red WiFi local a la que ambos dispositivos puedan conectarse
- Arduino IDE con soporte ESP32 y biblioteca DHT
- Python: pip install flask psutil requests (en el servidor)

FASE 1: FIRMWARE DEL NODO ESP32

Programar el ESP32 para que actúe como cliente HTTP que envía datos ambientales al servidor central.

Conexiones hardware: DHT22 -> GPIO4 (con pull-up 10k), LED -> GPIO2 (con resistencia 220ohm).

Firmware ESP32 (Arduino IDE):

```
// lab_integrador_esp32.ino
#include <WiFi.h>
#include <HTTPClient.h>
#include <DHT.h>
#include <ArduinoJson.h>  // instalar via Library Manager

const char* SSID      = "TU_RED_WIFI";
const char* PASSWORD  = "TU_PASSWORD";
const char* SERVER_IP = "192.168.1.XXX";  // IP del servidor Flask
const int   SERVER_PORT = 5000;
const char* NODE_ID   = "ESP32-SERVER-ROOM-01";

DHT dht(4, DHT22);
const int LED_PIN = 2;
const float TEMP_UMBRAL = 32.0;  // grados C

void setup() {
    Serial.begin(115200);
    pinMode(LED_PIN, OUTPUT);
    dht.begin();
    WiFi.begin(SSID, PASSWORD);
    Serial.print("Conectando WiFi");
    while (WiFi.status() != WL_CONNECTED) {
        delay(500); Serial.print(".");
        digitalWrite(LED_PIN, !digitalRead(LED_PIN));  // parpadeo
    }
    Serial.println("\n[OK] WiFi conectado: " + WiFi.localIP().toString());
    digitalWrite(LED_PIN, LOW);
}
```

```
void loop() {
    float temp = dht.readTemperature();
    float hum  = dht.readHumidity();
    if (isnan(temp) || isnan(hum)) {
        Serial.println("[ERROR] Sensor DHT22 no responde");
        delay(5000); return;
    }
    // Alerta visual
    if (temp > TEMP_UMBRAL) {
        for (int i = 0; i < 3; i++) {  // 3 destellos rapidos
            digitalWrite(LED_PIN, HIGH); delay(100);
            digitalWrite(LED_PIN, LOW);  delay(100);
        }
    }
    // Construir JSON
    StaticJsonDocument<200> doc;
    doc["node_id"]     = NODE_ID;
    doc["temperatura"] = round(temp * 10) / 10.0;
    doc["humedad"]     = round(hum * 10) / 10.0;
    doc["rssi_dbm"]    = WiFi.RSSI();
    doc["alerta"]      = (temp > TEMP_UMBRAL);
    doc["uptime_s"]    = millis() / 1000;
    String payload;
    serializeJson(doc, payload);
    // Enviar al servidor Flask
    HTTPClient http;
    String url = "http://" + String(SERVER_IP) + ":" + SERVER_PORT +
"/api/node";
    http.begin(url);
    http.addHeader("Content-Type", "application/json");
    int code = http.POST(payload);
    Serial.print("[POST] "); Serial.print(url);
    Serial.print(" -> HTTP "); Serial.print(code);
    Serial.print(" | T:"); Serial.print(temp,1);
    Serial.print("C H:"); Serial.print(hum,1); Serial.println("%");
    http.end();
    delay(30000);  // enviar cada 30 segundos
}
```

FASE 2: SERVIDOR FLASK CON API REST Y DASHBOARD

Instalar dependencias: 'pip install flask psutil requests' en la PC del servidor.

El servidor expone 3 endpoints: POST /api/node (recibe datos del ESP32), GET /api/metrics (devuelve métricas locales del hardware), GET / (dashboard HTML).

Servidor Flask (Python 3):

```
# server.py - Ejecutar con: python server.py
from flask import Flask, request, jsonify
import psutil, time, sqlite3, json
```

```
from datetime import datetime

app = Flask(__name__)
DB_FILE = 'monitoring.db'

# ─── Base de datos ──────────────────────────────────────
def init_db():
    with sqlite3.connect(DB_FILE) as conn:
        conn.execute('''CREATE TABLE IF NOT EXISTS node_data (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            timestamp TEXT, node_id TEXT,
            temperatura REAL, humedad REAL,
            rssi_dbm INTEGER, alerta INTEGER)''')
        conn.execute('''CREATE TABLE IF NOT EXISTS host_metrics (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            timestamp TEXT, cpu_pct REAL,
            ram_pct REAL, disco_pct REAL, temp_cpu REAL)''')

# ─── Endpoints API ──────────────────────────────────────
@app.route('/api/node', methods=['POST'])
def recibir_nodo():
    data = request.get_json()
    ts = datetime.now().isoformat()
    with sqlite3.connect(DB_FILE) as conn:
        conn.execute('INSERT INTO node_data VALUES (NULL,?,?,?,?,?,?)',
            (ts, data.get('node_id'), data.get('temperatura'),
             data.get('humedad'), data.get('rssi_dbm'), int(data.get('alerta',
False))))
    print(f'[{ts}] Nodo: {data}')
    return jsonify({'status': 'ok', 'timestamp': ts}), 200

@app.route('/api/metrics')
def metricas_host():
    cpu = psutil.cpu_percent(interval=1)
    ram = psutil.virtual_memory().percent
    disco = psutil.disk_usage('/').percent
    # Temperatura CPU (si disponible)
    temp_cpu = None
    try:
        temps = psutil.sensors_temperatures()
        if 'coretemp' in temps:
            temp_cpu = round(temps['coretemp'][0].current, 1)
        elif 'k10temp' in temps:
            temp_cpu = round(temps['k10temp'][0].current, 1)
    except: pass
    # Guardar historico
    ts = datetime.now().isoformat()
    with sqlite3.connect(DB_FILE) as conn:
        conn.execute('INSERT INTO host_metrics VALUES (NULL,?,?,?,?,?)',
                     (ts, cpu, ram, disco, temp_cpu))
    # Ultimos datos del ESP32
    node_latest = None
    with sqlite3.connect(DB_FILE) as conn:
        row = conn.execute(
```

```
            'SELECT temperatura,humedad,rssi_dbm,alerta,timestamp FROM
node_data ORDER BY id DESC LIMIT 1'
        ).fetchone()
        if row:
            node_latest = {'temperatura': row[0], 'humedad': row[1],
                           'rssi': row[2], 'alerta': bool(row[3]), 'ts':
row[4]}
    return jsonify({
        'timestamp': ts,
        'host': {'cpu_pct': cpu, 'ram_pct': ram, 'disco_pct': disco,
'temp_cpu': temp_cpu},
        'node': node_latest
    })

@app.route('/')
def dashboard():
    return '''<!DOCTYPE html>
<html><head><meta charset="utf-8"><title>Monitor TI</title>
<style>
  body{font-family:Arial,sans-
serif;background:#1a2f4a;color:#fff;margin:0;padding:20px}
  h1{color:#1abc9c;text-align:center}
  .grid{display:grid;grid-template-columns:repeat(auto-
fit,minmax(200px,1fr));gap:15px;margin:20px 0}
  .card{background:#2471a3;border-radius:12px;padding:20px;text-align:center}
  .val{font-size:42px;font-weight:bold;color:#1abc9c}
  .lbl{font-size:13px;color:#aaa;margin-top:5px}
  .alert{color:#e74c3c !important}
  #log{background:#0a1929;padding:10px;border-radius:8px;font-
family:monospace;font-size:12px;height:120px;overflow-y:auto}
</style></head>
<body>
<h1>Monitor de Infraestructura TI  |  Lab Integrador</h1>
<div class="grid">
  <div class="card"><div class="val" id="cpu">-</div><div class="lbl">CPU
%</div></div>
  <div class="card"><div class="val" id="ram">-</div><div class="lbl">RAM
%</div></div>
  <div class="card"><div class="val" id="disco">-</div><div class="lbl">Disco
%</div></div>
  <div class="card"><div class="val" id="tcpu">-</div><div class="lbl">Temp
CPU</div></div>
  <div class="card"><div class="val" id="tamb">-</div><div class="lbl">Temp
Ambiente (ESP32)</div></div>
  <div class="card"><div class="val" id="hum">-</div><div class="lbl">Humedad
(ESP32)</div></div>
</div>
<div id="log">Iniciando monitoreo...</div>
<script>
async function update(){
  try{
    const r=await fetch('/api/metrics'); const d=await r.json();
    document.getElementById('cpu').textContent  =
d.host.cpu_pct.toFixed(1)+'%';
    document.getElementById('ram').textContent  =
d.host.ram_pct.toFixed(1)+'%';
```

```
    document.getElementById('disco').textContent=
d.host.disco_pct.toFixed(1)+'%';
    document.getElementById('tcpu').textContent = d.host.temp_cpu ?
d.host.temp_cpu+'C' : 'N/D';
    if(d.node){
      const ta=document.getElementById('tamb');
      ta.textContent=d.node.temperatura+'C';
      ta.className='val'+(d.node.alerta?' alert':'');
      document.getElementById('hum').textContent=d.node.humedad+'%';
    }
    const log=document.getElementById('log');
    log.innerHTML=new Date().toLocaleTimeString()+' |
CPU:'+d.host.cpu_pct.toFixed(1)+'%
RAM:'+d.host.ram_pct.toFixed(1)+'%<br>'+log.innerHTML;
  }catch(e){console.error(e);}
}
update(); setInterval(update,5000);
</script></body></html>'''

if __name__ == '__main__':
    init_db()
    print('Servidor iniciando en http://0.0.0.0:5000')
    app.run(host='0.0.0.0', port=5000, debug=False)
```

PROCEDIMIENTO DE INTEGRACION

1. Instalar las dependencias Python en la PC: 'pip install flask psutil requests'.
2. Ejecutar el servidor Flask: 'python server.py'. Anotar la IP de la PC ejecutando 'ipconfig' (Windows) o 'ip addr' (Linux). Navegar a http://localhost:5000 para verificar el dashboard.
3. Configurar el firmware del ESP32: reemplazar SSID, PASSWORD y SERVER_IP con los valores correctos de tu red.
4. Cargar el firmware al ESP32. Abrir el Monitor Serial. Verificar que se conecta al WiFi y que los POST HTTP devuelven código 200.
5. Verificar en el dashboard web que los datos del ESP32 aparecen en las tarjetas 'Temp Ambiente' y 'Humedad'.
6. Simular carga en el servidor: ejecutar Cinebench o una compilación mientras el dashboard está abierto. Verificar que el % de CPU sube en tiempo real.
7. Simular alerta de temperatura: cubrir el sensor DHT22 con la mano. Cuando la temperatura supere el umbral, verificar que el LED del ESP32 parpadea y que la tarjeta de temperatura en el dashboard se vuelve roja.
8. Revisar la base de datos SQLite: 'sqlite3 monitoring.db' seguido de 'SELECT * FROM node_data ORDER BY id DESC LIMIT 10;' para ver el historial de mediciones.
9. Documentar con capturas de pantalla: el dashboard con datos reales, el monitor serial del ESP32 con los POST exitosos, y una consulta a la base de datos con datos históricos.

ENTREGABLES DEL LABORATORIO INTEGRADOR

- Firmware del ESP32 comentado y funcionando, con captura del monitor serial mostrando conexión WiFi-exitosa y al menos 5 POST HTTP exitosos.
- Servidor Flask ejecutando y recibiendo datos, con captura del dashboard en el navegador mostrando métricas reales de la PC y del ESP32.
- Captura de la base de datos SQLite con al menos 10 registros históricos de ambas tablas (node_data y host_metrics).
- Diagrama de arquitectura del sistema: muestra el ESP32, la red Wifi, el servidor Flask, la base de datos y el cliente del dashboard con las flechas de comunicación y los protocolos usados en cada enlace.
- Reporte de 2-3 páginas que incluya: descripción del sistema, tecnologías usadas de cada unidad del curso, problemas encontrados y como se resolvieron, y propuestas de mejora (ej: agregar graficas históricas, alertas por email, persistencia en la nube, segundo nodo sensor).
- Reflexión final (1 pagina): ¿Como se relaciona este laboratorio con los conceptos de arquitectura de la Unidad 1 (jerarquía de memoria, bus del sistema)? ¿Con los de la Unidad 2 (protocolos SPI/I2C/UART, SoC)? ¿Con los de la Unidad 3 (evaluación de hardware, infraestructura TI)?

PREGUNTAS DE REFLEXION FINAL

- ¿Qué rol juega el pipeline y el multicore del procesador de la PC (Unidad 1) cuando el servidor Flask atiende múltiples clientes simultáneamente?
- ¿Por qué el ESP32 usa WiFi (capa de aplicación HTTP) para comunicarse con el servidor en lugar de UART directo o I2C? ¿En qué escenario UART directo seria más eficiente?
- ¿Como se podría reemplazar el servidor Flask en la PC por un servicio en la nube (AWS IoT Core, Azure IoT Hub)? ¿Qué parte del código del ESP32 cambiaria?
- El sistema almacena datos en SQLite. ¿Que ocurrirá después de 1 año de operación con registros cada 30 segundos? ¿Cuántos MB ocupara la base de datos? ¿Como resolverías el crecimiento indefinido?
- Diseña una extensión del sistema para una empresa con 10 salas de servidores: ¿cuántos ESP32 necesitarías? ¿Como modificarías el servidor Flask para manejar múltiples nodos? ¿Qué información adicional seria útil monitorear?